

字节前端AI开发

一面（技术基础） · 1-6

1. 请简述一下 Promise 的工作机制，以及它与 async/await 的关系。
2. 在 AI 聊天场景中，流式输出（Streaming）通常是怎么实现的？
3. 如何处理前端大文本内容的展示，避免页面卡顿？
4. 请手写一个具有并发控制的请求函数。
5. CSS 如何实现一个类似 ChatGPT 的气泡对话框，要求宽度自适应且有最大宽度？
6. 谈谈你对 Prompt Engineering（提示工程）在前端开发中应用场景的理解。

二面（深入原理与项目） 1-8

1. React 的渲染机制中，如何优化 AI 聊天机器人这种频繁更新 state 的场景？
2. 详细说明一下如何实现 AI 自动滚动到底部，并处理用户手动向上滚动时的冲突？
3. 在前端实现 RAG（检索增强生成）流程中，前端通常承担哪些职责？
4. 谈谈 Web Worker 在 AI 前端应用中的具体使用场景。
5. 如何设计一个可扩展的 AI 组件库，支持多种模型和交互形态？
6. 请解释一下 Token 的概念，以及前端如何估算 Prompt 的 Token 长度？
7. 针对 AI 响应慢的问题，前端可以做哪些用户体验上的优化？
8. 手写题：实现一个简单的流式数据解析器，将 Uint8Array 转换为文本块。

三面（架构与综合能力） 1-5

1. 如果让你从零设计一个企业级的 AI 助手前端架构，你会考虑哪些核心模块？
2. 在 AI 业务中，如何定义并监控前端的性能指标？
3. 谈谈 AI Agent（智能体）在前端交互上与传统 Chatbot 有什么区别？
4. 面对大模型技术的快速迭代，作为前端工程师，你认为核心竞争力应该体现在哪里？
5. 聊聊你在过去一年中遇到的最难的 AI 相关前端问题，你是如何解决的？

一面（技术基础）

1. 请简述一下 Promise 的工作机制，以及它与 async/await 的关系。

难度：★

考点：JS 异步编程基础

答案要点：

Promise 是 JS 中处理异步操作的对象，代表一个未来可能完成或失败的操作及其结果值。

- 状态机机制：具有 Pending、Fulfilled、Rejected 三种状态，状态一旦改变不可逆。
- 链式调用：通过 .then() 和 .catch() 解决回调地狱，支持返回新的 Promise。
- 语法糖关系：async/await 是基于 Generator 和 Promise 的语法糖，使异步代码看起来像同步代码。
- 错误处理：async/await 使用 try-catch 捕获异常，比 Promise 的错误处理更符合直觉。

常见坑：

- 忘记在 async 函数中 await 导致返回的是 Promise 对象而非结果。
- 在 forEach 循环中使用 await 无法按预期顺序执行。

追问：

- 如果有多个并行的 AI 接口请求，你会用哪个 API 来处理？

2. 在 AI 聊天场景中，流式输出（Streaming）通常是怎么实现的？

难度：★★

考点：网络协议与流式传输

答案要点：

流式输出主要通过 SSE（Server-Sent Events）或 WebSocket 实现，目前主流 AI 接口多采用 SSE。

- SSE 原理：基于 HTTP 协议，服务端设置 Content-Type 为 text/event-stream，保持长连接。
- 单向通信：SSE 只支持服务端向客户端推送数据，适合 AI 这种一问一答且回复较长的场景。
- 浏览器 API：前端使用 EventSource 接口或 Fetch API 的 ReadableStream 进行解析。
- 优势：相比 WebSocket 更轻量，支持自动重连，且对现有 HTTP 基础设施兼容性好。

常见坑：

- 代理服务器（如 Nginx）开启了缓存导致流式效果失效。
- 忽略了 Fetch API 处理流式数据时需要手动解析 chunk。

追问：

- Fetch API 如何读取 ReadableStream？

3. 如何处理前端大文本内容的展示，避免页面卡顿？

难度：★★

考点：前端性能优化

答案要点：

处理大文本的核心思路是减少 DOM 节点的渲染压力和内存占用，通常采用按需加载或虚拟化技术。

- 虚拟滚动：只渲染可视区域内的文本行，对于超长对话记录非常有效。
- 分片渲染：利用 requestAnimationFrame 将长文本分批次插入 DOM。

- Web Worker：将复杂的文本处理、正则匹配或 Markdown 解析逻辑放在后台线程。
- 样式优化：避免在长文本容器上使用复杂的 CSS 选择器或频繁触发重排。

常见坑：

- 直接将数万字的文本赋值给 innerHTML 导致浏览器假死。
- 频繁更新 state 导致整个聊天列表重复渲染。

追问：

- 如果是实时生成的流式文本，虚拟滚动该如何计算高度？

4. 请手写一个具有并发控制的请求函数。

难度：★★

考点：手写代码能力、异步控制

答案要点：

并发控制是为了防止同一时间发送过多请求导致网络拥塞或触发服务端限流。

- 核心逻辑：维护一个执行队列和当前运行计数器。
- 递归执行：当一个任务完成后，自动从队列中取下一个任务执行。
- 返回结果：需要返回一个 Promise，在所有任务完成后 resolve 结果数组。

代码块

```
1  async function limitConcurrency(tasks, limit) {
2    const results = [];
3    const executing = new Set();
4    for (const task of tasks) {
5      const p = Promise.resolve().then(() => task());
6      results.push(p);
7      executing.add(p);
8      const clean = () => executing.delete(p);
9      p.then(clean).catch(clean);
10     if (executing.size >= limit) {
11       await Promise.race(executing);
12     }
13   }
14   return Promise.all(results);
15 }
```

常见坑：

- 无法正确处理任务失败的情况。
- 没有返回结果数组或顺序错乱。

追问：

- 如果任务有优先级，该如何修改？

5. CSS 如何实现一个类似 ChatGPT 的气泡对话框，要求宽度自适应且有最大宽度？

难度：★

考点：CSS 布局

答案要点：

利用块级元素的特性结合 max-width 和 flex 布局可以轻松实现。

- 容器布局：父容器使用 flex 布局，通过 flex-direction 控制左右对齐。
- 气泡样式：设置 display: inline-block 或 width: fit-content 使宽度随内容自适应。
- 边界约束：设置 max-width 限制最大宽度，配合 word-break: break-word 防止长单词溢出。
- 装饰细节：使用 border-radius 和伪元素 (::after/::before) 制作小三角。

常见坑：

- 忘记设置 word-break 导致长 URL 撑破容器。
- 在 flex 容器下，子元素默认 stretch 导致气泡占满全行。

追问：

- 如何实现气泡内的代码块横向滚动而不撑开气泡？

6. 谈谈你对 Prompt Engineering（提示工程）在前端开发中应用场景的理解。

难度：★

考点：AI 基础意识

答案要点：

提示工程在前端不仅用于对话，更用于结构化数据的生成和交互逻辑的辅助。

- 结构化输出：通过 Prompt 要求模型返回 JSON 格式，方便前端直接解析渲染。
- 少样本学习（Few-shot）：在 Prompt 中提供示例，确保 AI 生成的代码或内容符合特定 UI 规范。
- 角色设定：为 AI 设定特定角色（如代码审查专家），提高其在特定场景下的回答质量。
- 动态上下文：前端根据用户当前的操作路径动态拼接上下文，提高 AI 响应的准确性。

常见坑：

- 认为 Prompt 越长越好，忽略了 Token 成本和模型窗口限制。
- 没考虑到模型输出的不稳定性，缺乏兜底逻辑。

追问：

- 如何防止用户输入的 Prompt 注入攻击？

二面（深入原理与项目）

1. React 的渲染机制中，如何优化 AI 聊天机器人这种频繁更新 state 的场景？

难度：★★

考点：框架原理与性能优化

答案要点：

AI 聊天场景中，流式数据的每一字更新都会触发渲染，优化核心在于减少不必要的组件重绘。

- 局部状态下放：将流式文本的状态限制在最小的 MessageItem 组件内，避免父级 List 整体重绘。
- 使用 Memo：对历史消息组件使用 React.memo，只有当消息 ID 或内容变化时才重新渲染。
- 批量更新：利用 React 18 的自动批处理特性，或者在非 React 环境下手动节流更新频率。
- 引用稳定：确保事件处理函数使用 useCallback 包裹，避免子组件因 props 变化而失效。

常见坑：

- 在 Context 中存放流式文本，导致整个应用所有组件频繁刷新。
- 忽略了 key 的正确使用，使用 index 作为 key 导致列表更新性能低下。

追问：

- 如果实现打字机效果，你会如何控制渲染频率？

2. 详细说明一下如何实现 AI 自动滚动到底部，并处理用户手动向上滚动时的冲突？

难度：★★

考点：交互细节处理

答案要点：

这是一个典型的交互细节问题，需要通过监听滚动事件和判断滚动位置来决定是否自动跳转。

- 自动滚动：在 useEffect 监听消息内容变化，使用 scrollToView 或设置 scrollTop 触底。
- 冲突检测：监听容器的 onWheel 或 onScroll 事件，判断用户是否向上滚动（scrollTop + clientHeight < scrollHeight）。
- 状态锁定：当用户手动向上滚动时，设置一个“锁定”标志位，停止自动滚动。
- 恢复机制：提供一个“回到最新消息”的悬浮按钮，点击后恢复自动滚动并重置标志位。

常见坑：

- 自动滚动过于频繁导致用户无法正常阅读上方内容。
- 在图片或代码块加载完成前计算高度不准确，导致滚动不到位。

追问：

- 如何平滑地处理滚动动画？

3. 在前端实现 RAG（检索增强生成）流程中，前端通常承担哪些职责？

难度：★★★

考点：AI 架构理解

答案要点：

RAG 流程中，前端不仅是展示层，还参与了预处理、检索触发和结果重排的交互。

- 文档预处理：在前端对用户上传的文件进行分段（Chunking）预览或简单的格式清洗。
- 向量化触发：调用 Embedding 接口将查询转化为向量（有时在后端完成，但前端需处理状态）。
- 引用展示：将模型生成的回答与检索到的原始片段进行关联展示，支持点击跳转溯源。
- 反馈闭环：提供点赞/点踩功能，收集用户对检索准确性的反馈，用于后续优化。

常见坑：

- 忽略了长文档分段后的上下文丢失问题。
- 溯源链接失效或与当前展示内容不匹配。

追问：

- 前端如何处理 Embedding 向量这种高维数据？

4. 谈谈 Web Worker 在 AI 前端应用中的具体使用场景。

难度：★★

考点：多线程编程

答案要点：

Web Worker 能够将耗时计算从主线程剥离，保证 AI 交互界面的流畅度。

- 文本解析：在后台线程将复杂的 Markdown 或 LaTeX 转换为 HTML 字符串。
- 本地模型运行：如使用 WebLLM 在浏览器端运行小参数模型，必须在 Worker 中执行。
- 向量相似度计算：在本地知识库场景下，在前端进行简单的余弦相似度计算。
- 数据脱敏：在上传 Prompt 前，在 Worker 中进行敏感词过滤或正则匹配。

常见坑：

- 频繁在主线程和 Worker 间传递大量数据导致序列化开销过大。
- 忘记处理 Worker 的异常情况导致后台进程静默崩溃。

追问：

- 如何在 Worker 中引用第三方库？

5. 如何设计一个可扩展的 AI 组件库，支持多种模型和交互形态？

难度：★★★

考点：组件设计模式

答案要点：

设计 AI 组件库需要解耦视图层、逻辑层和协议层。

- 协议抽象：定义统一的 ChatMessage 格式，屏蔽不同模型接口（OpenAI/Anthropic）的字段差异。
- Headless 逻辑：提取 useChat 等自定义 Hook，处理流式解析、重试、历史记录管理等逻辑。

- 插槽机制：提供渲染自定义消息类型的能力（如图片、图表、代码块、Action 按钮）。
- 插件系统：支持注入工具调用（Tool Use）的 UI 渲染器，动态展示 AI 执行过程。

常见坑：

- 组件与特定后端接口强耦合，导致切换模型时需要重写 UI。
- 样式过于固定，难以适配不同产品的视觉风格。

追问：

- 如何处理 AI 输出中的多模态内容（如一边说话一边画图）？

6. 请解释一下 Token 的概念，以及前端如何估算 Prompt 的 Token 长度？

难度：★★

考点：AI 基础原理

答案要点：

Token 是模型处理文本的最小单位，通常一个单词或一个汉字对应 1-2 个 Token。

- 计算必要性：前端需要预估 Token 以防止超过模型的上下文窗口限制，并计算成本。
- 算法工具：通常使用 tiktoken（OpenAI）或 gpt-tokenizer 等库进行计算。
- 估算规则：英文通常 4 字符一个 Token，中文约 0.75 汉字一个 Token，但具体取决于编码器。
- 截断策略：当 Token 接近上限时，前端需实现滑动窗口算法，剔除最早的历史消息。

常见坑：

- 简单用字符串长度代替 Token 计数，导致计算偏差巨大。
- 在主线程同步计算长文本的 Token，导致界面掉帧。

追问：

- 为什么不同的模型 Token 计数结果不一样？

7. 针对 AI 响应慢的问题，前端可以做哪些用户体验上的优化？

难度：★★

考点：用户体验设计

答案要点：

AI 生成具有天然的延迟，前端需要通过视觉反馈和预处理来缓解焦虑。

- 骨架屏与状态反馈：在请求发出后立即展示 Loading 或思考中的动画。
- 乐观更新：用户发送消息后立即渲染到列表，而不是等待接口响应。
- 流式渲染：采用打字机效果实时展示结果，让用户感知到系统正在工作。
- 预加载与预测：根据用户输入的前缀，提前预热连接或加载相关资源。

常见坑：

- 没有任何反馈导致用户以为点击没生效，重复发送请求。

- 流式输出速度过快，导致页面剧烈跳动无法阅读。

追问：

- 如果流式输出中断了，前端该如何优雅地提示？

8. 手写题：实现一个简单的流式数据解析器，将 Uint8Array 转换为文本块。

难度：★★

考点：二进制处理、流式解析

答案要点：

处理流式数据时，需要处理字符被截断的情况（尤其是多字节的中文）。

- TextDecoder：使用 TextDecoder 的 { stream: true } 模式自动处理跨 chunk 的字符拼接。
- 缓冲区：如果需要解析特定格式（如 JSON 行），需要维护一个字符串缓冲区。
- 结束符处理：识别 \n\n 或 data: 等协议前缀。

代码块

```
1  async function parseStream(reader) {
2    const decoder = new TextDecoder();
3    let done = false;
4    while (!done) {
5      const { value, done: doneReading } = await reader.read();
6      done = doneReading;
7      const chunk = decoder.decode(value, { stream: !done });
8      console.log("Received chunk:", chunk);
9      // 进一步处理 SSE 格式数据...
10   }
11 }
```

常见坑：

- 没设置 { stream: true } 导致中文乱码。
- 忽略了最后一块数据的处理。

追问：

- 如何解析 SSE 协议中的 data: 字段？

三面（架构与综合能力）

1. 如果让你从零设计一个企业级的 AI 助手前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统架构设计

答案要点：

企业级架构需要考虑稳定性、安全性和可扩展性，采用分层设计。

- 通信层：封装 SSE/WebSocket/Fetch，支持统一的拦截器、重试机制和多模型切换。
- 状态管理层：管理长对话上下文、Token 计数、历史记录持久化（IndexedDB）。
- 渲染引擎：支持 Markdown、LaTeX、代码高亮及多模态组件的动态加载。
- 安全与监控：实现 Prompt 敏感词过滤、前端水印、性能指标（首字延迟、全文本耗时）监控。

常见坑：

- 缺乏持久化机制，页面刷新后对话丢失。
- 架构设计过于臃肿，导致简单场景开发效率低。

追问：

- 如何实现对话记录的本地搜索？

2. 在 AI 业务中，如何定义并监控前端的性能指标？

难度：★★★

考点：性能监控与数据分析

答案要点：

除了传统的 FP/LCP，AI 业务更关注交互的实时性和响应质量。

- TTFT（Time to First Token）：从发送请求到接收到第一个字符的时间，衡量响应速度。
- TPS（Tokens Per Second）：流式输出的速率，衡量生成过程是否平滑。
- 请求成功率：监控模型幻觉导致的报错或网络中断。
- 用户满意度指标：点赞率、修改率、对话轮数等业务指标。

常见坑：

- 只监控接口耗时，忽略了前端解析和渲染的开销。
- 监控数据量过大，没有做合理的抽样。

追问：

- 如何监控流式输出过程中的卡顿？

3. 谈谈 AI Agent（智能体）在前端交互上与传统 Chatbot 有什么区别？

难度：★★★

考点：AI 前沿趋势

答案要点：

Agent 强调的是任务拆解和工具调用，前端需要展示其“思考”和“执行”的过程。

- 过程可视化：展示 Agent 的思考链（CoT），如“正在搜索...”、“正在计算...”。
- 交互式反馈：当 Agent 需要调用工具（如发邮件）时，前端需弹出确认框或表单供用户干预。

- 动态 UI (Generative UI)：根据 Agent 的执行结果，动态生成特定的交互组件而非纯文本。
- 多状态管理：Agent 可能同时运行多个子任务，前端需要管理复杂的并行状态。

常见坑：

- 过程展示过于死板，用户看不懂 Agent 在干什么。
- 缺乏人工干预入口，导致 Agent 错误执行危险操作。

追问：

- 你认为 Generative UI 的核心难点在哪里？

4. 面对大模型技术的快速迭代，作为前端工程师，你认为核心竞争力应该体现在哪里？

难度：★★

考点：职业思考与价值观

答案要点：

前端工程师的价值在于连接 AI 能力与最终用户，创造极致的交互体验。

- 交互定义能力：如何利用 AI 改变传统的表单/点击交互，设计更自然的对话或多模态界面。
- 工程化落地：解决 AI 带来的性能、安全、稳定性问题，将 Demo 转化为生产级产品。
- 领域知识结合：将前端技术与模型能力结合（如 WebGPU 跑本地模型、Canvas 辅助 AI 绘图）。
- 敏捷学习：快速掌握 LangChain、LlamaIndex 等新工具，并将其转化为前端可用的能力。

常见坑：

- 陷入纯工具使用的陷阱，忽略了前端基础技术的深度。
- 盲目追求新技术，不考虑业务场景的实际收益。

追问：

- 你最近在关注 AI 领域的哪个新技术？它对前端有什么影响？

5. 聊聊你在过去一年中遇到的最难的 AI 相关前端问题，你是如何解决的？

难度：★★★

考点：项目复盘与问题解决

答案要点：

此题旨在考察候选人的实战经验和解决复杂问题的逻辑。

- 背景描述：简洁交代项目背景及遇到的具体技术挑战（如长对话内存溢出、流式渲染卡顿等）。
- 分析过程：说明如何定位问题（如使用 Chrome DevTools 分析内存泄漏）。
- 方案对比：列举尝试过的多种方案，并说明最终选择某方案的原因。
- 最终结果：量化的改进效果（如首屏时间降低 50%，支持了 10 万字长文本）。

常见坑：

- 描述过于笼统，没有具体的技术细节。

- 解决方案听起来太简单，体现不出深度。

追问：

- 如果现在让你重新做一遍，你会怎么改进？